

RAPPORT DE TRAVAUX PRATIQUES

Snort

Détection d'intrusion réseau (IDS)

*Configuration et expérimentation d'un système
de détection d'intrusion sous Linux*

Environnement Labtainer — Naval Postgraduate School
Center for Cybersecurity and Cyber Operations
National Science Foundation Award No. 1438893

1. Présentation

Ce travail pratique présente l'utilisation du système Snort afin de fournir une détection d'intrusion dans un environnement Linux. Les étudiants y configurent des règles Snort simples et expérimentent avec un système de détection d'intrusion réseau (IDS — Intrusion Detection System).

Le laboratoire a été développé dans le cadre Labtainer par la Naval Postgraduate School (Center for Cybersecurity and Cyber Operations), grâce au financement du National Science Foundation Award No. 1438893. Ce travail est dans le domaine public et ne peut pas être protégé par des droits d'auteur.

Ressources complémentaires

- CNIL — <https://all-it-network.com/snort/>
- Manuel officiel Snort — <https://www.snort.org/documents/snort-users-manual>
- Port mirroring avec iptables — <https://www.it-connect.fr/port-mirroring-sous-linux-avec-iptables/>

2. Environnement du laboratoire

Le laboratoire fonctionne dans l'environnement Labtainer, disponible sur <http://my.nps.edu/web/c3o/labtainers>. Ce site comprend des liens vers une machine virtuelle préconstruite sur laquelle Labtainer est installé, mais Labtainer peut être exécuté sur n'importe quel hôte Linux prenant en charge Docker.

À partir du répertoire labtainer-student, on démarre le laboratoire avec les commandes suivantes :

```
export DISPLAY=:0  
labtainer snort
```

Un lien vers le manuel du laboratoire s'affiche alors automatiquement.

3. Configuration du réseau

Le laboratoire comprend plusieurs ordinateurs en réseau (cf. Figure 1). Lorsque le laboratoire démarre, plusieurs terminaux virtuels sont disponibles, chacun étant connecté à un composant différent.

3.1 Rôle de la passerelle (gateway)

La passerelle est configurée avec iptables pour utiliser la NAT, traduisant les adresses sources du trafic à partir des adresses IP internes (par exemple 192.168.2.1) vers notre adresse externe, c'est-à-dire 203.0.113.10.

Les règles iptables de la passerelle redirigent également le trafic web (ports 80 et 443) vers le composant serveur web, en traduisant l'adresse de destination visible depuis l'extérieur en l'adresse interne du serveur web.

La passerelle est aussi configurée pour refléter (mirroring) le trafic qui entre dans la passerelle via le lien 203.0.113.10 ou via le lien vers le serveur web. Ce trafic miroir est acheminé vers le composant snort. Cette mise en miroir permet au composant snort de reconstruire les sessions TCP entre le serveur web et les adresses externes.

3.2 Composants du réseau

Composant	Adresse IP	Description
gateway	192.168.1.10 / 192.168.2.10 / 192.168.3.10 / 203.0.113.10	Passerelle NAT et miroir de trafic vers Snort
snort	192.168.3.1	Hôte exécutant Snort et Wireshark
web server	192.168.1.2	Serveur Apache (HTTP/HTTPS) pour www.example.com
ws2 (Mary)	192.168.2.1	Station de travail interne (Firefox, nmap)
remote gateway	172.16.0.10 / 203.0.113.20	Passerelle simulant un réseau distant
remote ws	172.16.0.1	Station de travail distante (Firefox, nmap)

3.3 Détail des hôtes

Le composant snort comprend l'utilitaire Snort IDS ainsi que Wireshark, qui aide à observer le trafic mis en miroir vers le composant snort.

Le serveur web exécute Apache et est configuré pour prendre en charge SSL pour les pages web du domaine www.example.com.

Le composant remote_ws comprend le navigateur Firefox ainsi qu'un fichier /etc/hosts local qui mappe www.example.com à l'adresse externe de la passerelle (203.0.113.10). La station de travail interne ws2 comprend également Firefox et une entrée dans /etc/hosts pour www.example.com. Les deux stations de travail disposent en outre de l'utilitaire nmap.

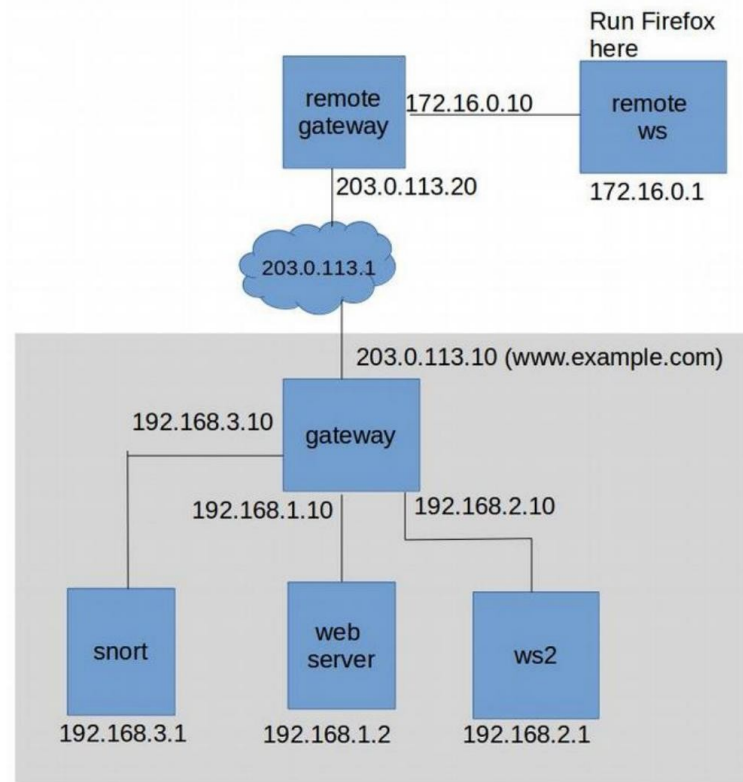


Figure 1 — Topologie du réseau pour le laboratoire Snort

4. Tâches de laboratoire

Il est supposé que l'étudiant a déjà reçu un enseignement (ou effectué une étude indépendante) sur le fonctionnement de base de Snort, ainsi que sur les objectifs généraux et les mécanismes de la détection d'intrusion dans le réseau.

Avant toute manipulation, il convient de passer en revue la topologie du réseau. On s'intéresse en particulier aux paramètres iptables sur la passerelle, visibles en examinant les commandes du fichier `/etc/rc.local`. Ces commandes définissent les traductions NAT et — c'est essentiel pour ce laboratoire — la mise en miroir (mirroring) du trafic vers le composant snort.

Ressource complémentaire sur le port mirroring avec iptables : <https://www.it-connect.fr/port-mirroring-sous-linux-avec-iptables/>

4.1 Démarrer et arrêter Snort

L'utilitaire Snort est installé sur le composant snort. Le répertoire d'accueil comprend un script `start_snort.sh` qui démarre l'utilitaire en mode de détection d'intrusion réseau et affiche les alertes sur la console. Pour ce laboratoire, on démarre Snort avec :

```
./start_snort.sh
```

Lorsque l'on souhaite arrêter Snort (par exemple pour ajouter ou modifier des règles), il suffit d'utiliser la combinaison CTRL+C.

4.2 Règles Snort préconfigurées

L'utilitaire Snort comprend un ensemble de règles préconfigurées qui génèrent des alertes pour des activités réseau suspectes connues. La configuration du composant snort est globalement celle qui existe immédiatement après l'installation initiale de l'utilitaire.

Pour observer un exemple de règles préconfigurées, on effectue un scan nmap de `www.example.com` depuis la station de travail distante :

```
sudo nmap www.example.com
```

On note alors les alertes affichées sur la console snort. Les règles qui déclenchent ces alertes sont visibles, comme toutes les règles, dans le répertoire `/etc/snort/rules/`.

4.3 Écrire une (mauvaise) règle simple

Les règles personnalisées sont généralement ajoutées au fichier `/etc/snort/rules/local.rules`. Pour illustration, écrivons une règle qui génère une alerte pour chaque paquet d'un flux TCP :

```
alert tcp any any -> any any (msg : "TCP detected" ; sid:00002 ;)
```

Cette règle peut être lue ainsi : « Générer une alerte chaque fois qu'un paquet TCP provenant de n'importe quelle adresse sur n'importe quel port est envoyé à n'importe quelle adresse sur n'importe quel port, afficher le message 'TCP detected', et donner à la règle l'identifiant 00002. »

On redémarre ensuite Snort, puis on teste cette règle en démarrant Firefox sur la station de travail distante :

```
firefox www.example.com
```

Observation

Comme on peut le constater, la règle ainsi écrite produit une avalanche d'informations inutiles : chaque paquet TCP déclenche une alerte. Cela illustre la nécessité de cibler précisément le trafic à surveiller. On arrête donc Snort et on supprime cette règle.

4.4 Règle personnalisée pour le trafic CONFIDENTIEL

Dans le navigateur Firefox, qui affiche la page web de `www.example.com`, on souhaite afficher une page web non publiée dont nous savons qu'elle existe sur le site. En particulier, l'entreprise aurait placé des plans confidentiels à l'adresse :

```
www.example.com/plan.html
```

On y jette un coup d'œil.

On ajoute ensuite une règle au fichier `local.rules` de Snort qui générera une alerte chaque fois que le texte « CONFIDENTIAL » est envoyé sur le réseau. On peut se référer au manuel Snort (section Payload Detection Rule Options) ou aux règles existantes pour comprendre comment qualifier les alertes en fonction de leur contenu. On veille à inclure le mot « CONFIDENTIAL » dans le message d'alerte et à donner à la règle un sid unique. Après avoir ajouté la règle, on redémarre Snort.

Sur le navigateur de la station distante, on efface l'historique (Menu / Préférences / Sécurité et confidentialité), puis on rafraîchit la page `plan.html`. On observe alors une alerte dans la console Snort.

Règle utilisée

```
alert tcp any any -> any any (msg:"CONFIDENTIAL detected";
content:"CONFIDENTIAL"; nocase; sid:10000002;)
```

Remarque

Le mot-clé `nocase` signifie « insensible à la casse » : la règle se déclenche que le mot apparaisse en majuscules, minuscules ou en casse mixte (CONFIDENTIAL, confidential, Confidential, etc.).

4.5 Effets du chiffrement

De retour au navigateur Firefox, on efface à nouveau l'historique du navigateur. On modifie ensuite l'URL pour utiliser la fonction SSL du serveur web :

```
https://www.example.com/plan.html
```

Voit-on une nouvelle alerte Snort ? Pourquoi ?

Analyse

En HTTPS, tout le trafic HTTP est chiffré, ce qui inclut la requête HTTP complète ainsi que les données échangées. Étant donné que Snort ne voit que le trafic chiffré, il ne pourra pas retrouver les phrases définies dans nos signatures à l'intérieur des paquets. Ces phrases n'existent que dans le trafic déchiffré, auquel Snort n'a pas accès.

Référence : <https://security.stackexchange.com/a/216475>

Solution envisageable

Une solution à ce problème consiste à utiliser un reverse proxy (proxy inverse) placé devant le serveur web. Ce reverse proxy prend en charge le trafic entrant et gère lui-même les connexions SSL. Le serveur web ne reçoit alors que du trafic HTTP en clair, et le trafic sortant du serveur web peut ainsi être reflété vers l'IDS.

Cette solution ne sera cependant pas mise en œuvre dans le cadre de ce laboratoire.

4.6 Surveillance du trafic interne

On se place sur le composant ws2 (la station de Mary) et on lance nmap :

```
sudo nmap www.example.com
```

Que voit-on sur le composant snort ? L'alerte ICMP PING NMAP apparaît-elle ? Pourquoi ?

Analyse

On ne voit que les alertes liées aux réponses au ping envoyées depuis le serveur web. On ne voit aucune alerte ayant pour source le PC de Mary, puisqu'aucun trafic provenant du réseau de Mary n'est, à ce stade, envoyé à l'IDS (Snort).

Pour corriger ce point, on se rend sur le composant passerelle et on modifie le script /etc/rc.local afin que le trafic de la station de travail de Mary soit reflété dans le composant snort. On ajoute la ligne suivante à la section du fichier qui définit la mise en miroir des paquets :

```
iptables -t mangle -A PREROUTING -i $lan2 -j TEE --gateway 192.168.3.1
```

On exécute ensuite le script afin de remplacer les règles iptables par les nouvelles :

```
sudo /etc/rc.local
```

On redémarre Snort, puis on relance nmap depuis l'ordinateur ws2 de Mary. Les alertes ICMP PING NMAP sont à présent visibles dans la console Snort, pour les deux origines (interne et externe).

4.7 Distinguer le trafic par adresse

On lance Firefox sur l'ordinateur ws2 de Mary afin de visualiser le plan d'affaires confidentiel :

```
firefox www.example.com/plan.html
```

On observe alors la console Snort. Une alerte CONFIDENTIAL est levée — ce qui n'est pas souhaité. En effet, les esprits vifs de la startup doivent pouvoir consulter leur plan d'affaires confidentiel sans que les alertes IDS ne se déclenchent. En revanche, ils veulent surveiller les ordinateurs internes pour détecter tout trafic suspect, par exemple les scans nmap.

L'objectif de cette tâche est donc d'ajuster la règle Snort de telle sorte que l'alerte CONFIDENTIAL ne se déclenche que lorsque le plan est accessible par des adresses extérieures au site.

Structure d'une règle Snort

Si l'on examine les règles trouvées dans le répertoire /etc/snort/rules, on constate qu'elles ont la forme générale suivante :

```
alert <protocole> <adr_source> <src_port> -> <adr_dest> <port_dest> (options  
entre parenthèses)
```

Les règles Snort comprennent deux champs d'adresse : `adr_source` et `adr_dest`, qui désignent respectivement l'origine et la destination du paquet. Chaque adresse peut être :

- une adresse IP unique (par exemple 192.168.1.2) ;
- le mot-clé `any` pour s'appliquer à toutes les adresses ;
- une adresse réseau au format CIDR. Par exemple, 192.168.2.0/24 représente le réseau de classe C 192.168.2.0 avec 24 bits dans le masque réseau.

Point clé sur la NAT

En raison de l'utilisation de la NAT, tout le trafic provenant du serveur web et destiné à une adresse externe aura pour adresse de destination la passerelle (192.168.1.10), tandis que le trafic web destiné aux utilisateurs internes aura une adresse de destination correspondant à l'utilisateur interne. Cette distinction permet de différencier les accès internes des accès externes.

Objectifs à atteindre

1. L'accès externe au plan d'affaires génère une alerte ;
2. L'accès interne au plan d'affaires ne génère pas d'alerte ;
3. L'utilisation externe ou interne de nmap génère une alerte ICMP NMAP PING.

Chacun de ces critères doit être testé au cours d'une seule session Snort : toute modification d'une règle Snort ou de la mise en miroir des ports impose de recommencer l'ensemble des tests.

Règle finale

```
alert tcp 192.168.1.2 80 -> 192.168.1.10 any (msg:"CONFIDENTIAL detected";  
content:"CONFIDENTIAL"; nocase; sid:10000003;)
```

Cette règle ne se déclenche que pour le trafic émis par le serveur web (192.168.1.2:80) vers la passerelle (192.168.1.10), c'est-à-dire vers un client externe (post-NAT). Les accès depuis ws2/Mary, dont la destination reste interne (192.168.2.1), ne déclenchent pas l'alerte.

4.8 Vérification de l'avancement

Une commande de vérification est mise à disposition de l'étudiant afin de valider l'avancement du laboratoire :

```
checkwork
```

5. Soumission

Après avoir terminé le laboratoire, on se place dans le terminal du système Linux qui a été utilisé pour démarrer le laboratoire et on saisit la commande :

```
stoplab snort
```

À l'arrêt du laboratoire, le système affiche le chemin vers les résultats compressés du laboratoire sur le système Linux. Ce fichier doit être fourni à l'instructeur, par exemple via la plateforme Sakai.

Vérification côté instructeur

Pour l'instructeur, la vérification s'effectue via :

```
cd ../labtainer-instructor
gradelab snort
```

Résultats attendus (grille de notation)

Voici un exemple de sortie obtenue à l'issue de la vérification :

```
| snort_local_con | snort_remote_co | snort_remote_fi | snort_local_fir | proper_config | snort_local_nma |
| ===== Y | ===== Y | ===== Y | ===== Y | ===== Y | ===== Y |
```

Figure 2 — Sortie de la commande gradelab snort (tous les critères validés)

Le détail des critères de notation est repris dans le tableau ci-dessous :

Critère	Description	Statut
snort_local_con	Détection CONFIDENTIAL — accès local	Y
snort_remote_co	Détection CONFIDENTIAL — accès distant	Y
snort_remote_fi	Filtrage des alertes — accès distant	Y
snort_local_fir	Filtrage des alertes — accès local	Y
proper_config	Configuration correcte de la mise en miroir	Y
snort_local_nma	Détection nmap depuis le poste interne	Y

6. Conclusion

Ce travail pratique a permis de prendre en main l'outil Snort dans un environnement réseau réaliste comportant des hôtes internes, un serveur web exposé via NAT et des postes externes. Les différentes étapes ont successivement abordé :

- le démarrage et l'arrêt de Snort en mode IDS sur le composant dédié ;
- l'exploration des règles préconfigurées et la lecture des alertes générées par un scan nmap ;

- l'écriture d'une première règle (trop générique) afin d'illustrer la nécessité de cibler le trafic ;
- la création d'une règle ciblant la chaîne « CONFIDENTIAL » dans le trafic HTTP en clair ;
- la mise en évidence d'une limite fondamentale de la détection par signature : l'incapacité de Snort à inspecter le trafic chiffré (HTTPS) ;
- la configuration du port mirroring iptables pour permettre la surveillance du trafic interne ;
- l'affinage des règles à l'aide des champs adresse source / destination pour distinguer le trafic interne du trafic externe, en exploitant les effets de la NAT.

Ce laboratoire illustre à la fois la puissance et les limites des IDS basés sur signature : ils sont efficaces sur du trafic en clair et bien ciblés, mais deviennent aveugles face au chiffrement de bout en bout. Dans un déploiement réel, le placement de l'IDS dans la topologie réseau (en amont ou en aval d'un reverse proxy gérant la terminaison SSL) constitue donc un choix architectural majeur.